earnings-edge

Architecture, explained without jargon

What is it?

earnings-edge is a live website that, for each of India's 500 largest listed companies, shows how the share price has historically behaved around its quarterly earnings results. If you look up a stock, you can see what usually happens the day after that company reports, what happens over the next week, and whether large institutional traders have been buying or selling it recently.

It is **not** a stock-tip site and it does not predict the future. It is a reference tool that turns twelve years of price data and thousands of past earnings events into plain-English base rates: *this stock has ended higher five days after earnings 7 times out of the last 12*. The audience is a retail investor who wants to make decisions from history instead of hype.

Who uses it?

Anyone with a browser. The site is public at earnings-edge-omega.vercel.app. No sign-in. Someone types "RELIANCE" into the search box, and within a second they get the reliance page: today's price, a hit-rate summary, a chart of every past reaction, a table of large recent trades, and a screener that lets them filter the whole 500 by growth or momentum.

The three pieces

Think of the project as three layers that each do one job and pass their output to the next.

1. The collectors (data ingestion)

These are small Python programs that go out to the internet every night and gather fresh data from stock-exchange websites: today's closing prices, any large trades that were reported, foreign-investor cash flows, and each company's latest quarterly earnings numbers. They handle everything that goes wrong on the way — rate limits, timeouts, bad data — and clean the output before storing it. There are seven of them, one for each kind of data.

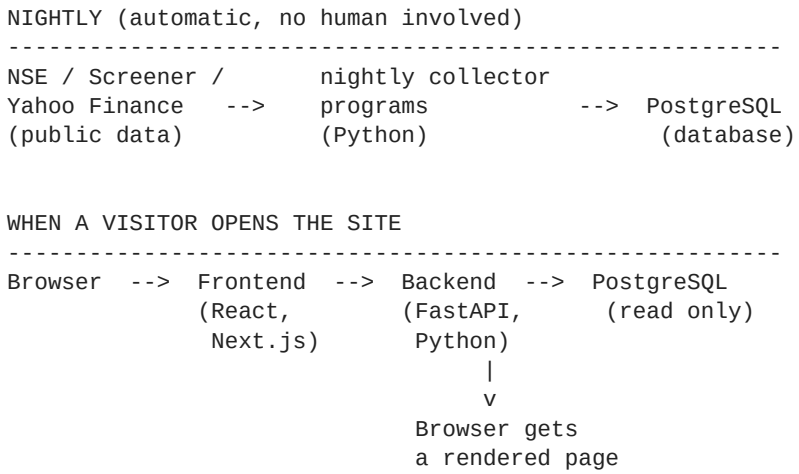
2. The database (memory)

All the collected data lives in a PostgreSQL database. That's about a million price rows, six thousand earnings events, and their computed reactions (±1 day, ±5 day price moves) sitting in structured tables. Nothing on the site is generated on the fly — every number a visitor sees is either read directly from this database or computed once and cached.

3. The website (display)

The website has two halves. A **backend** written in Python (FastAPI) takes requests like "give me RELIANCE's data" and returns clean JSON. A **frontend** written in TypeScript (Next.js and React) turns that JSON into the pages, charts, and tables you actually see in the browser. They talk to each other over the internet just like any two web services.

How they talk to each other



The arrows only ever go one way per request. The nightly programs write to the database. The website only reads from it. Nothing a visitor does can change the data, so the site is safe to leave open to the world.

Where the pieces live

Each layer runs on a different free hosting service, picked because they each specialise in exactly one thing:

Piece	Runs on	What it does
Database	Neon (Singapore)	Stores the data. Sleeps when idle.
Backend	Railway (San Francisco)	Runs the FastAPI server 24/7.
Frontend	Vercel (global CDN)	Serves the website to any browser.
Nightly jobs	GitHub Actions	Runs the collectors on a schedule.

All four are on free tiers. Total monthly cost of running the site: **0 USD**.

What runs automatically

The whole point of building it this way is that once it is deployed, it maintains itself. Every weekday evening, without anyone touching it, this sequence runs:

- Refresh the list of Nifty 500 companies (in case any got added or dropped).
- Pull today's closing prices for all 500 stocks.
- Refresh the split-adjusted price series from Yahoo Finance so any recent stock split doesn't fake a price gap.
- Pull today's large-trade (bulk and block) reports from the exchange.
- Pull today's foreign and domestic institutional cash flow numbers.
- Update delivery percentages (how much of today's volume was serious buyers vs day-traders).
- Take a snapshot of options prices for the F&O stocks (used to compute volatility rank over time).
- Compute the reactions for any newly announced earnings, and once a week, refresh the earnings numbers themselves from Screener.in.

If any step fails — a website is temporarily down, an API rate-limits us — the remaining steps still run. Each step logs its outcome to the database so failures are visible the next morning. The site never goes down because of a single flaky upstream source.

What actually happens when you open a stock page

To make the architecture concrete, here is the sequence of events between typing `/stock/RELIANCE` into the URL bar and seeing the page in front of you — usually inside one second.

1. Your browser sends a request to Vercel's servers.
2. Vercel returns the compiled React page, which starts loading in your browser.
3. The page immediately asks the FastAPI backend on Railway for five things in parallel: RELIANCE's basic info, its earnings history, its base-rate distributions, its positioning signals, and its similar past setups.
4. Railway forwards each of those requests as SQL queries to the Neon database in Singapore.
5. Neon returns the rows. Railway shapes them into clean JSON. Vercel returns that JSON to your browser.
6. React fills in the tables, draws the charts with Recharts, and renders the plain-English "7 out of 12 past results ended higher" summary.

Why this shape?

Splitting the work into **collectors, database, backend, frontend** means each piece can fail, be redeployed, or be replaced without touching the others. If the collector for foreign investor data breaks tomorrow, the website still runs on yesterday's numbers. If the frontend needs a redesign, the backend and database stay put. If a new data source becomes available, only a new collector needs to be written.

It is the same architecture used by real companies at much larger scale — the pieces just happen to fit on free hosting tiers here, and the single engineer maintaining it is one person on a laptop.

— Built and maintained by Sricharan D A · github.com/Char1an/earnings-edge